# Passe

Controle de acesso por QR Code e NFC para catracas. Roteiro de apresentação da Sprint 1 (ABP 4DSM 2026-2, FATEC Jacareí).

## 1. O problema

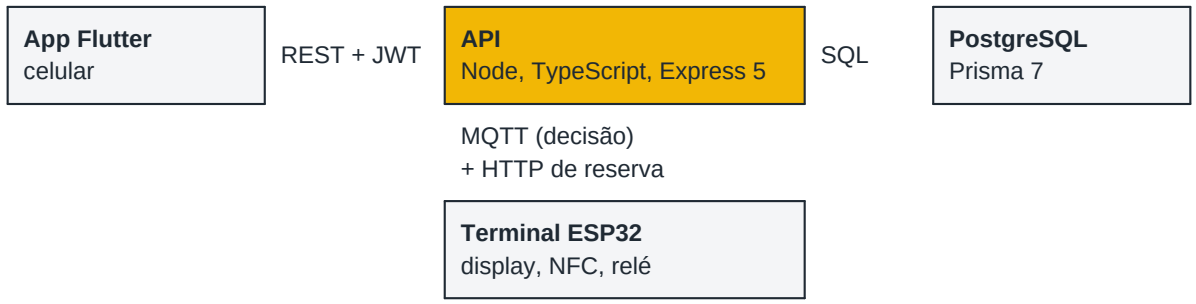
Um condomínio ou prédio precisa controlar quem passa em cada catraca, em qual horário, quantas vezes e se a pessoa está mesmo ali. Hoje isso depende de portaria manual ou de crachás que podem ser emprestados. O Passe resolve isso com o celular e uma tag NFC.

## 2. A ideia em uma analogia

> **Pense num cinema.** A **bilheteria** é a **API**: sabe quem comprou ingresso, para qual sessão e quantas vezes pode entrar. O **ingresso** é a **autorização**: vale para uma catraca, num período, num horário e com um número máximo de usos. O **funcionário na porta** é o **terminal (ESP32)**: ele não decide nada, só mostra o código e abre a porta quando a bilheteria manda. O **celular do morador** é a pessoa chegando com o ingresso.

O funcionário da porta não sabe quem tem ingresso: ele pergunta à bilheteria e obedece. Essa separação é a decisão central da arquitetura. **Toda a inteligência fica no servidor**, e o hardware é simples e trocável.

## 3. Arquitetura




| Peça | Tecnologia | Papel |
|---|---|---|
| **App** | Flutter | O morador lê o QR; a equipe administra |
| **API** | Node 24, TypeScript, Express 5, Prisma 7 | Valida tudo e decide |
| **Banco** | PostgreSQL | Usuários, locais, autorizações, histórico |
| **Mensageria** | MQTT (Mosquitto) | Entrega a decisão ao terminal na hora |
| **Terminal** | ESP32 | Mostra o QR, lê NFC, abre a catraca |

## 4. O fluxo do QR

> **O QR é uma senha de cofre que muda sozinha**, como o token do aplicativo do banco.

1. O terminal pede um QR à API. Ele **troca a cada 10 s** e **vale 30 s**.
2. O morador lê o QR com o app. O app envia o QR, a localização e o login dele.
3. A API verifica: a assinatura é verdadeira e o QR não venceu? O usuário está ativo e tem autorização para *aquela* catraca? Está no horário e ainda tem usos? Está a menos de **100 m** do terminal? O QR nunca foi usado?
4. A API manda **ABRIR** ou **NEGAR** ao terminal por MQTT, e o terminal aciona o relé.

**Por que o QR muda?** Se alguém tirar uma foto, a foto vale pouco: em 30 s ele venceu e, como é de uso único, só funciona uma vez. **Por que a distância?** Sem ela, o QR poderia ser enviado por mensagem e usado de longe. Com o GPS, o celular precisa estar perto da catraca.

## 5. O fluxo do NFC

É mais direto: o terminal lê a tag e manda o número dela à API, que responde na hora com ABRIR ou NEGAR. Não usa MQTT, porque a resposta volta na própria chamada.

## 6. Por que MQTT e não só HTTP?

> **HTTP é ligar para a bilheteria de tempos em tempos perguntando "alguma novidade?". MQTT é a bilheteria te chamar no rádio assim que há uma novidade.**

O MQTT entrega a decisão em tempo real. Como o rádio pode falhar, existe um plano B: o terminal também consulta a API a cada 2 s. Se o MQTT cair, a catraca continua funcionando, só um pouco mais devagar.

Detalhe de projeto: com o broker fora do ar, a API **descarta** a decisão em vez de guardá-la. Assim a catraca não abre atrasada quando o rádio voltar.

## 7. Como o ESP32 conversa com a API

Todas as chamadas HTTP levam os cabeçalhos **X-Terminal-Id** e **X-Terminal-Key**. A chave é gerada no cadastro do terminal e gravada no firmware.

| O quê | Como | Frequência |
|---|---|---|
| **Pegar o QR** | GET /terminal/qr-token | A cada 10 s |
| **Avisar que está vivo** | POST /terminal/heartbeat | A cada 30 s |
| **Tag NFC lida** | POST /terminal/nfc com o UID | A cada leitura |
| **Receber decisão do QR** | MQTT, tópico passe/terminais/{id}/decisoes (QoS 1) | Tempo real |
| **Reserva se o MQTT cair** | GET /terminal/decisions?apos={seq} | A cada 2 s |

Sem sinal por mais de 90 s, o terminal aparece como offline para o administrador.

### O que é o firmware

É o programa gravado dentro do ESP32, escrito em C++ no ambiente Arduino (Arduino IDE ou PlatformIO). Ele faz o que o simulador web já faz hoje: conecta no Wi-Fi, desenha o QR, assina o MQTT, aciona o relé, lê a tag NFC e manda o heartbeat. **O simulador é a especificação do firmware.**

## 8. Decisões de segurança

- **QR de uso único, curto e assinado:** a assinatura impede falsificação.
- **Chave por terminal:** a API guarda só o hash. Se um terminal for roubado, troca-se só a chave dele.
- **Perfis:** administrador, controlador de acesso, morador e visitante. O morador recebe 403 nas rotas da equipe.
- **Histórico completo:** todo acesso, liberado ou negado, fica registrado com o motivo.

## 9. O que já está pronto

| Parte | Situação |
|---|---|
| **Backend** | 7 módulos (auth, users, locations, authorizations, access, events, terminals). 78 testes automatizados passando. |
| **Banco** | PostgreSQL com 10 modelos Prisma, migration e seed de demonstração. |
| **MQTT** | Notificador MQTT implementado, com consulta HTTP de reserva. |
| **Simulador do terminal** | Página web que faz o papel do ESP32: QR rotativo, MQTT, NFC e um celular de teste. |
| **CI/CD** | GitHub Actions: lint, tipos, testes, build e imagem Docker em cada PR. |
| **Docker** | Banco, broker e API sobem com um comando. |
| **App Flutter** | Login, sessão e os 4 menus por perfil. A maioria das opções ainda mostra "tela em construção". |

## 10. O que falta (roadmap com issues abertas)

| Falta | Issue no GitHub |
|---|---|
| Telas do app: scanner de QR e resultado do acesso | #6 |
| ESP32 alcançar a API e o broker na rede | #7 |
| Cadastro do terminal real e chave do firmware | #8 |
| Definir o hardware (display, leitor NFC, relé) | #9 |
| Segurança do MQTT: senha, ACL e TLS | #10 |
| Firmware do ESP32 | Depende do hardware (#9) |
| Deploy em nuvem (Render) | Depois do protótipo local |

Apresente o que falta como **roadmap com issues já abertas**: isso mostra planejamento, não problema. A issue #5 (validar o fluxo no simulador) já está concluída.

## 11. Demonstração sugerida

| Passo | O que fazer | Resultado esperado |
|---|---|---|
| 1 | Abrir o simulador em http://localhost:3000/simulador/ | O QR aparece e troca sozinho |
| 2 | Usar o celular de teste perto da catraca | Acesso liberado |
| 3 | Ler o mesmo QR de novo | Negado: QR já utilizado |
| 4 | Mudar o celular de teste para "a 1 km" | Negado: longe do terminal |
| 5 | Passar uma tag NFC | Liberado |

A demonstração mostra segurança, regras de negócio e tempo real, sem depender de hardware.

## 12. Perguntas que o professor pode fazer

**Por que o ESP32 não decide sozinho?** Porque trocar uma regra de acesso exigiria regravar todos os terminais, e um terminal roubado carregaria as regras consigo. Com a decisão no servidor, o hardware não guarda segredo além da própria chave.

**E se a internet cair?** O MQTT cai, mas o terminal usa a consulta HTTP. Se a API inteira cair, nenhuma decisão é tomada. Esse é um risco conhecido, que tratamos na fase de nuvem.

**O broker é seguro?** Ainda não: está sem autenticação, ACL e TLS. Para o protótipo na rede local é aceitável, e isso está registrado na issue #10.

**Por que a localização vem do celular?** Porque o terminal não sabe onde está o celular. Limitação: um app adulterado pode mentir a posição. Isso reduz o risco, mas não o elimina.

## 13. Preparação para quarta-feira

- Suba a API antes da apresentação, para o limite de 30 s do QR valer, e deixe o Docker já aberto.
- Ensaie a demonstração uma vez antes.
- Tenha o roadmap (issues #5 a #10) aberto no GitHub.